

LCD Display (SPI)

LCD Display (SPI)

1. Learning Objectives
 - SPI Protocol
 - SPI Hardware Interface
 - SPI Advantages
 - SPI Disadvantages
2. Hardware Setup
3. Experimental Steps
 1. Open SYSCONFIG Configuration Tool
 2. SPI Parameter Configuration
 3. Pin Parameter Configuration
 - LCD Screen
 - External Flash
 3. SPI Protocol Usage
 4. Write Program
 5. Compile
4. Program Analysis
5. Experimental Phenomenon

1. Learning Objectives

1. Learn and understand basic SPI communication knowledge.
2. Read data from flash and display it on the onboard LCD screen.

SPI Protocol

SPI protocol (Serial Peripheral Interface) is a commonly used synchronous serial communication protocol designed for high-speed, short-distance communication between microcontrollers and external devices. It supports full-duplex communication, meaning data can be sent and received simultaneously between master and slave devices. It is commonly used for memory chips, sensors, display drivers, wireless modules, and other devices.

SPI Hardware Interface

SPI protocol typically uses four lines for data transmission:

- **SCLK (Serial Clock):** Clock signal generated by the master device for synchronizing communication.
- **MOSI (Master Output Slave Input):** Data sent from master device to slave device.
- **MISO (Master Input Slave Output):** Data sent from slave device to master device.
- **SS/CS (Slave Select):** Master device activates the target slave device through chip select signal.

SPI Advantages

- Fast communication speed, suitable for high bandwidth applications.
- Supports full-duplex transmission, improving efficiency.
- Simple hardware implementation, direct interface.
- Supports multiple slave device connections.

SPI Disadvantages

- Lack of standardized error detection mechanism.
- Master device pin count increases with the number of slave devices.
- Communication distance is limited, usually used for onboard communication.

2. Hardware Setup

W25Q32 is a common serial flash device using SPI (Serial Peripheral Interface) protocol. It features high-speed read, write, and erase functions and is widely used in embedded systems, storage devices, routers, and other high-performance electronic devices. W25Q32 has a capacity of 32 Mbit (i.e., 4 MB). The numbers in the model represent different capacity options, such as W25Q16, W25Q64, W25Q128, etc., to meet different application requirements.

W25Q32 chip memory is allocated by sector and block. Specifically:

- **Sector:** Each sector is 4KB in size.
- **Block:** Each block contains 16 sectors, meaning one block is 64KB in size.

These storage unit structures make data management and storage operations more flexible, especially suitable for embedded systems and storage applications that require frequent read/write operations.

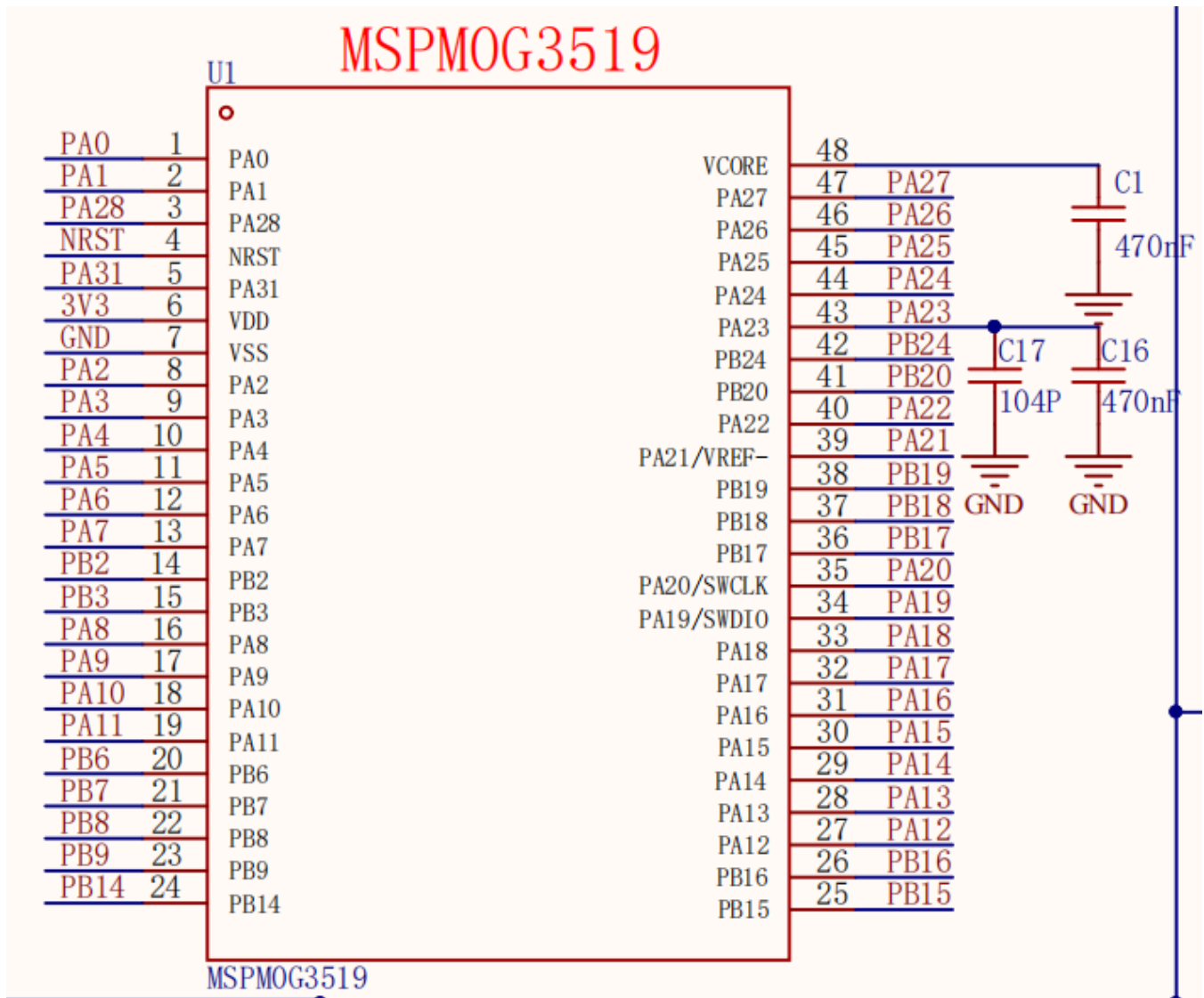
We use hardware SPI to drive W25Q32, so we need to determine whether the pins we set have hardware SPI peripheral interface. In the datasheet, PB14~PB17 can be multiplexed as SPI1's 4 communication lines.

PZ PIN	PIN NAME/ IOMUX REG/ IOMUX ADDR	SIGNAL NAME	IOMUX PF	SIGNAL TYPE	BUFFER TYPE
48	PB14 PINCM31 0x40428078	PB14	1	IO	SDIO (standard)
		SPI1_CS3	2	IO	
		SPI1_POCI	3	IO	
		TIMG12_C1	4	IO	
		TIMA0_C0	5	IO	
		TIMG8_IDX	6	I	
		SPI0_CS3	7	IO	
49	PB15 PINCM32 0x4042807c	PB15	1	IO	SDIO (standard)
		UART7_TX	2	O	
		SPI1_PICO	3	IO	
		UART3_CTS	4	I	
		TIMG8_C0	5	IO	
		I2C2_SCL	7	IOD	
		TIMG7_C0	11	IO	
50	PB16 PINCM33 0x40428080	PB16	1	IO	SDIO (standard)
		UART7_RX	2	I	
		SPI1_SCK	3	IO	
		UART3_RTS	4	O	
		TIMG8_C1	5	IO	
		I2C2_SDA	7	IOD	
		TIMG7_C1	11	IO	
73	PB17 PINCM43 0x404280a8	PB17	1	IO	SDIO (standard)
		UART7_TX	2	O	
		SPI0_PICO	3	IO	
		I2C0_SCL	4	IOD	
		TIMA0_C2	5	IO	
		TIMG0_C0	6	IO	
		SPI1_CS1	7	IO	
		UART4_TX	8	O	
		TIMG14_C2	9	I	
		TIMA1_C0	11	IO	

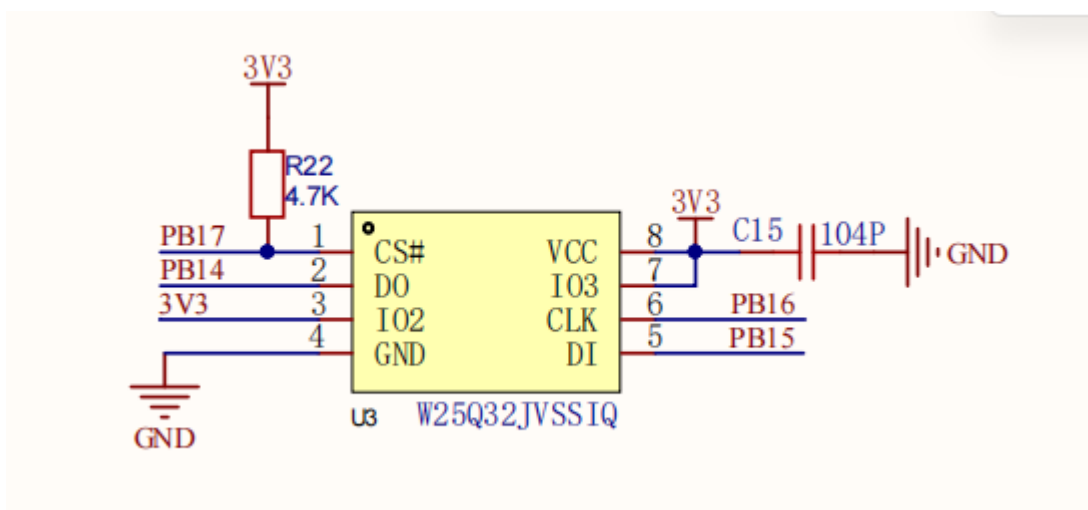
AI_4	(Non-IOMUX 1) 0	A
COMP1_IN2-	(Non-IOMUX 2) 0	A

This course does not require additional hardware devices, directly using the onboard LCD screen and external storage flash on the MSPM0G3519 mainboard.

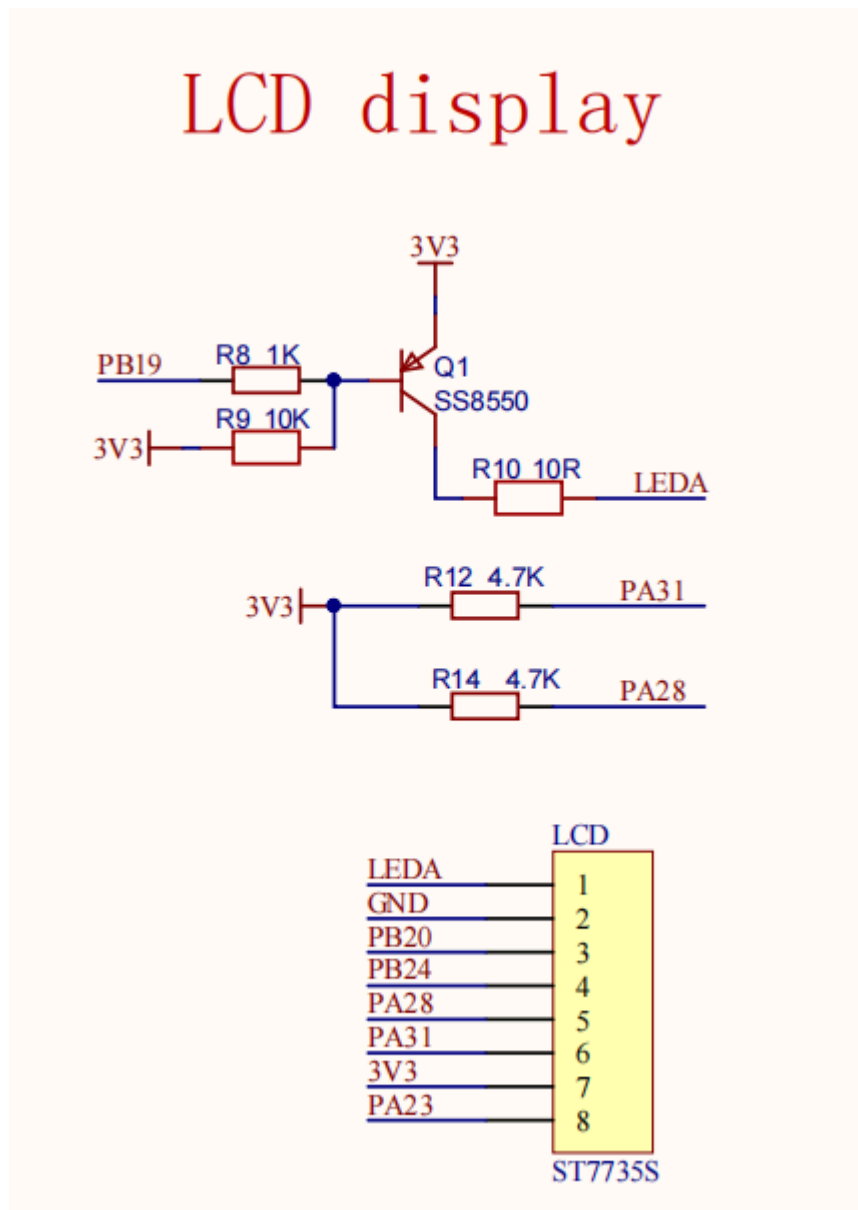
MSPM0G3519 Main Control Diagram:



W25Q32 Partial Schematic:



LCD Screen Partial Schematic:



3. Experimental Steps

Here we use the template project we provided for introduction.

Place the template project in the SDK root directory and extract it. I renamed it as the 11_SPI folder.

新加卷 (D:) > ti > mspm0_sdk_2_02_00_05

名称	修改日期	类型	大小
01_LED	2024/11/19 11:42	文件夹	
02_Delay	2024/11/19 18:29	文件夹	
03_KEY	2024/11/19 20:09	文件夹	
04_ExtInterrupt	2024/11/19 21:15	文件夹	
05_UART	2024/11/20 19:41	文件夹	
06_Timer	2024/11/20 20:28	文件夹	
07_PWM	2024/11/21 15:59	文件夹	
08_ADC	2024/11/21 18:37	文件夹	
09_DMA	2024/11/21 19:48	文件夹	
10_I2C	2024/11/26 14:08	文件夹	
11_SPI	2024/11/26 14:20	文件夹	
docs	2024/11/14 15:05	文件夹	
examples	2024/11/14 15:05	文件夹	
kernel	2024/11/14 15:09	文件夹	

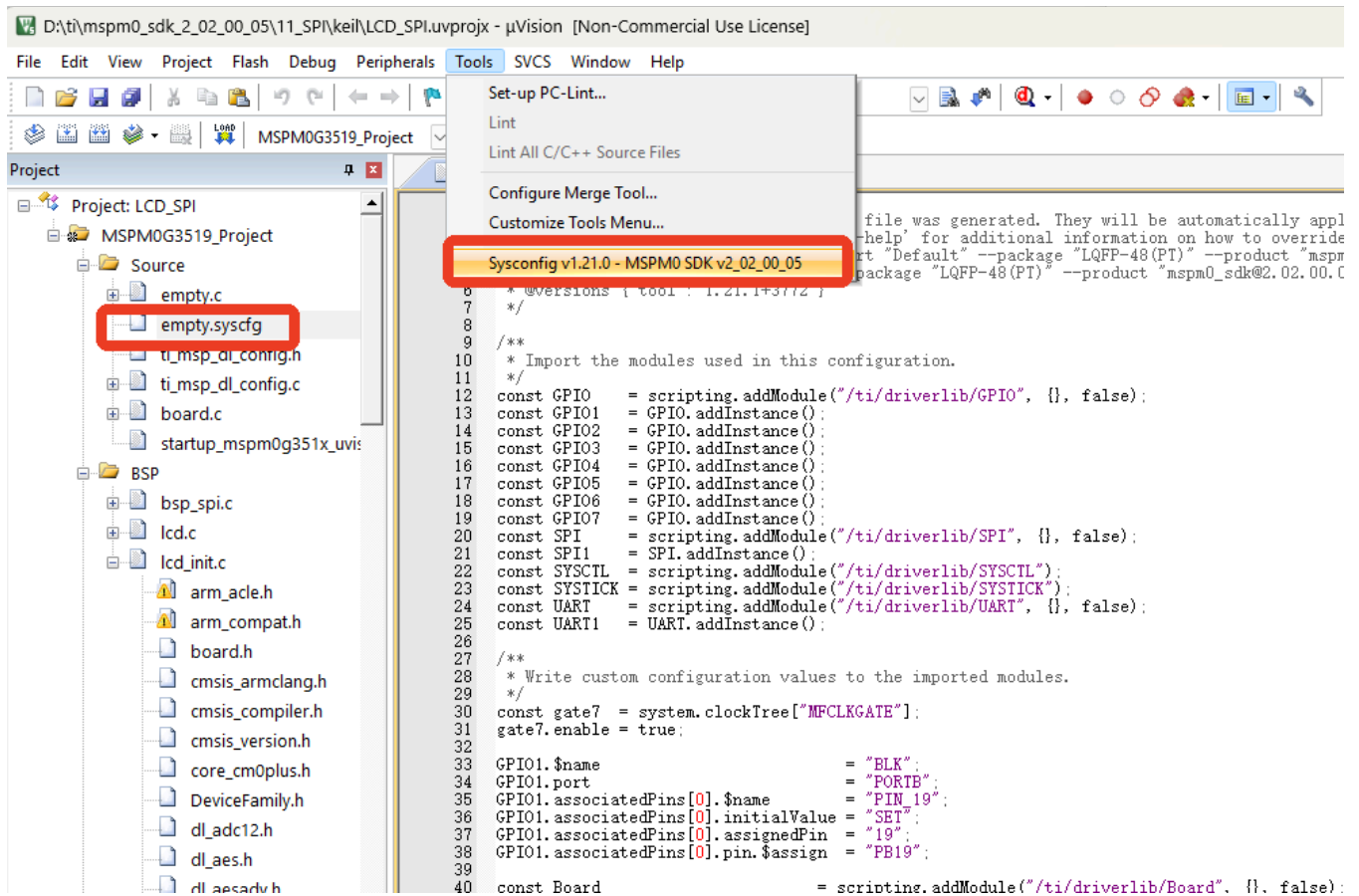
1. Open SYSCONFIG Configuration Tool

Then open the project.

此电脑 > 软件 (D:) > ti > mspm0_sdk_2_02_00_05 > 11_SPI > keil >

名称	修改日期	类型	大小
Listings	2026/8/31 17:57	文件夹	
Objects	2026/8/31 17:57	文件夹	
AllHeader.h	2023/3/6 11:37	H 文件	1 KB
EventRecorderStub.scvd	2024/11/29 19:06	SCVD 文件	1 KB
LCD_SPI.uvguix.Administrator	2026/8/19 18:38	ADMINISTRAT...	176 KB
LCD_SPI.uvoptx	2026/8/19 18:36	UVOPTX 文件	21 KB
LCD_SPI.uvprojx	2026/8/19 18:35	uVision5 Project	20 KB
mspm0g3519.sct	2024/1/25 11:42	Windows Script...	1 KB
startup_mspm0g351x_uvision.s	2026/8/19 18:35	S 文件	10 KB

Open the empty.syscfg file in Keil's main interface. With the empty.syscfg file open, then open the SYSCONFIG GUI interface.



Find the SPI section on the left, click to enter, and add SPI peripheral.

Type Filter Text...

Software > SPI

MSPM0 DRIVER LIBRARY ...

SYSTEM (9)

Board 1/1 ✓ (+)
Configuration NVM (+)
DMA (+)
GPIO 7 ✓ (+)
MATHACL (+)
RTC (+)
SYSCTL 1/1 ✓ (+)
SYSTICK 1/1 ✓ (+)
WWDG (+)

ANALOG (6)

ADC12 (+)
COMP (+)
DAC12 (+)
GPAMP (+)
OPA (+)
VREF (+)

COMMUNICATIONS (6)

I2C (+)
I2C - SMBUS (+)
MCAN (+)

SPI 1/2 ✓ (+)
UART 1/4 ✓ (+)
UART - LIN (+)

TIMERS (6)

SPI (1 of 2 Added) ⓘ

+ ADD

REMOVE ALL

✓ SPI

Peripheral does not retain register contents in STOP or STANDBY modes. User should take care to save and restore register configuration in application. See Retention Configuration section for more details.

Name SPI

Selected Peripheral SPI1

Quick Profiles

SPI Profiles Custom

Basic Configuration

SPI Initialization Configuration

Mode Select Controller

Clock Configuration

Target Bit Rate (Hz) 16000000

Calculated Bit Rate 16000000.00

Calculated Error (%) 0

2. SPI Parameter Configuration

✓ SPI

Peripheral does not retain register contents in STOP or STANDBY modes. User should take care to save and restore register configuration in application. See Retention Configuration section for more details.

Name

SPI

Selected Peripheral

SPI1

Quick Profiles

SPI Profiles

Custom

Basic Configuration

SPI Initialization Configuration

Mode Select

Controller

Clock Configuration

Target Bit Rate (Hz)

16000000

Calculated Bit Rate

16000000.00

Calculated Error (%)

0

Frame Format

Motorola 4-wire

Clock Polarity

Low

Phase

Data captured on first clock edge

Frame Size (bits)

8

Bit Order

MSB

SPI Controller Basic Configuration

Chip Select

CS1

Advanced Configuration

Parity

Disabled

RX FIFO Threshold Level

RX FIFO contains >= 2 entries

TX FIFO Threshold Level

TX FIFO contains <= 2 entries

Communication Direction

PICO and POCI

Enable Packing

☐

Parameter Description:

Mode Select: SPI master/slave mode selection. Select Controller here (master)

Target Bit Rate (Hz): SPI frequency configuration. Enter 16000000, SPI clock maximum 16MHZ

Calculated Bit Rate: Actual frequency calculated by software based on configuration

Calculated Error (%): Calculated frequency error

Frame Format: SPI mode. Select Motorola 4-wire, 4-wire mode

Clock Polarity: SPI clock polarity configuration. Select Low, polarity is low

Phase: SPI phase configuration. Configure as Data captured on first clock edge, first phase capture data

Frame Size (bits): Data bit size. Configure as 8-bit data frame

Bit Order: Data order. Configure as MSB, high bit first

Chip Select: Chip select line configuration. Select CS1. This is not important here, later we select software method to control CS, not hardware

Parity: Whether to enable SPI check. Select Disabled, do not enable

RX FIFO Threshold Level: Configure RX FIFO interrupt trigger level. Configure as RX FIFO contains >= 2 entries

TX FIFO Threshold Level: Configure TX FIFO interrupt trigger level. Configure as TX FIFO contains <= 2 entries

Communication Direction: SPI communication direction. Configure as PICO and POI, both send and receive

Enable Packing: Whether to enable 32-bit sending method. Do not enable here

PinMux Peripheral and Pin Configuration			^
SPI Peripheral	SPI1	▼	🔒
SPI SCLK (Clock)	PB16/4	▼	🔒
SPI PICO (Peripheral In, Controller Out)	PB15/3	▼	🔒
SPI POI (Peripheral Out, Controller In)	PB14/2	▼	🔒
SPI CS1 (Chip Select 1)	Any(PA27/31)	▼	

CS pin configure randomly first, later we use software method for CS.

3. Pin Parameter Configuration

Find the GPIO section on the left, click to enter, and add several groups of GPIOs.

SysConfig - D:\ti\mspm0_sdk_2_02_00_05\10_I2C\empty.syscfg

FILE ABOUT RESTART

Type Filter Text...

Software > GPIO

GPIO (3 Added)

ADD REMOVE ALL

LED1 KEY I2C

Name LED1

Port PORTB

Port Segment Any

Group Pins

1 added

ADD REMOVE ALL

PIN_2

Name PIN_2

Direction Output

Initial Value Cleared

IO Structure Any

Digital IOMUX Features

Assigned Port PORTB

Assigned Port Segment Any

Assigned Pin 2

Interrupts/Events

PinMux Peripheral and Pin Configuration

Other Dependencies

Problems

There are no problems in the current design.

Generated Files

Filter: all

File name	Category	Include in build
ti_msp_dl_config.c	MSPM0 Driver Library	☒
ti_msp_dl_config.h	MSPM0 Driver Library	☒
Event.dot	MSPM0 Driver Library	☒
empty.syscfg	Configuration Script	☐

4 Total Files

MSPM0G3519 (Device)
LQFP-48(PT) (Package)

SWITCH

Pin Available

Pin Assigned

Warning

Power

Ground

Fixed (N/A)

GPIO Used 8/44

LCD Screen

Backlight BLK:

GPIO (6 Added)

+ ADD
REMOVE ALL

✓

BLK

✓

CS

✓

DC

✓

RES

✓

MOSI

✓

SCLK

Name

BLK

Port

PORTB

Port Segment

Any

Group Pins

^

1 added

+ ADD
REMOVE ALL

✓

PIN_19

Name

PIN_19

Direction

Output

Initial Value

Set

IO Structure

Any

Digital IOMUX Features

▼

Assigned Port

PORTB

Assigned Port Segment

Any

Assigned Pin

19

Interrupts/Events

▼

CS, DC, RES, MOSI, SCLK are as follows:

GPIO (6 Added) ⓘ

ADD REMOVE ALL

BLK

CS

DC

RES

MOSI

SCLK

Name

CS

Port

PORTA

Port Segment

Any

Group Pins

1 added

ADD REMOVE ALL

PIN_23

Name

PIN_23

Direction

Output

Initial Value

Cleared

IO Structure

Any

Digital IOMUX Features

Assigned Port

PORTA

Assigned Port Segment

Any

Assigned Pin

23

Interrupts/Events

GPIO (6 Added) ⓘ

ADD REMOVE ALL

BLK

CS

DC

RES

MOSI

SCLK

Name

DC

Port

PORTB

Port Segment

Any

Group Pins

1 added

ADD REMOVE ALL

PIN_24

Name

PIN_24

Direction

Output

Initial Value

Cleared

IO Structure

Any

Digital IOMUX Features

Assigned Port

PORTB

Assigned Port Segment

Any

Assigned Pin

24

Interrupts/Events

GPIO (6 Added) ⓘ

ADD REMOVE ALL

BLK

CS

DC

RES

MOSI

SCLK

Name

RES

Port

PORTB

Port Segment

Any

Group Pins

1 added

ADD REMOVE ALL

PIN_20

Name

PIN_20

Direction

Output

Initial Value

Cleared

IO Structure

Any

Digital IOMUX Features

Assigned Port

PORTB

Assigned Port Segment

Any

Assigned Pin

20

Interrupts/Events

GPIO (6 Added) ⓘ

ADD REMOVE ALL

BLK

CS

DC

RES

MOSI

SCLK

Name

SCLK

Port

PORTA

Port Segment

Any

Group Pins

1 added

ADD REMOVE ALL

PIN_31

Name

PIN_31

Direction

Output

Initial Value

Cleared

IO Structure

Any

Digital IOMUX Features

Assigned Port

PORTA

Assigned Port Segment

Any

Assigned Pin

31

Interrupts/Events

GPIO (6 Added) ⓘ

ADD REMOVE ALL

BLK

CS

DC

RES

MOSI

SCLK

Name

MOSI

Port

PORTA

Port Segment

Any

Group Pins

1 added

ADD REMOVE ALL

PIN_28

Name

PIN_28

Direction

Output

Initial Value

Cleared

IO Structure

Any

Digital IOMUX Features

Assigned Port

PORTA

Assigned Port Segment

Any

Assigned Pin

28

Interrupts/Events

External Flash

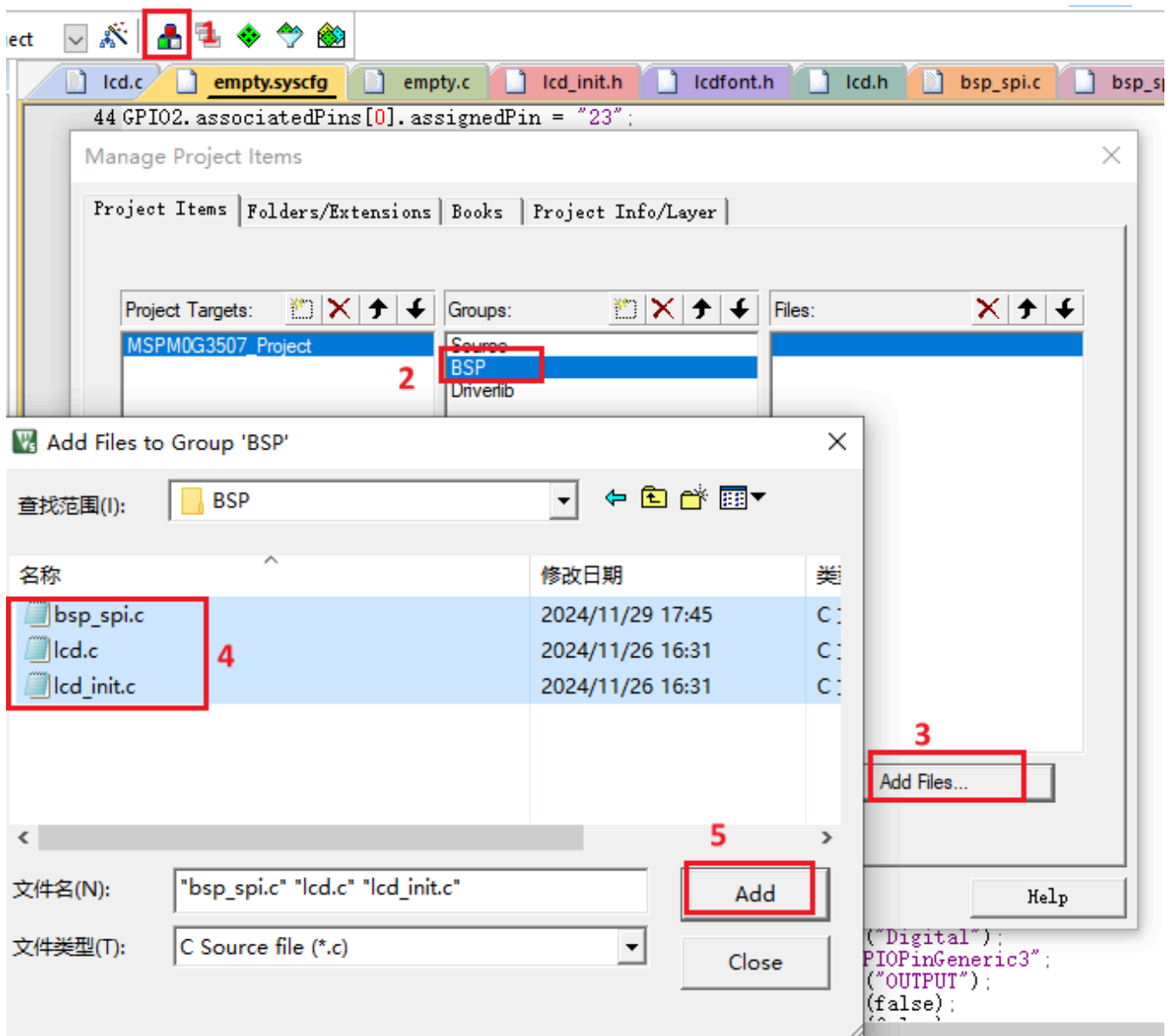
CS1

;) > ti > mspm0_sdk_2_02_00_05 > 11_SPI > BSP

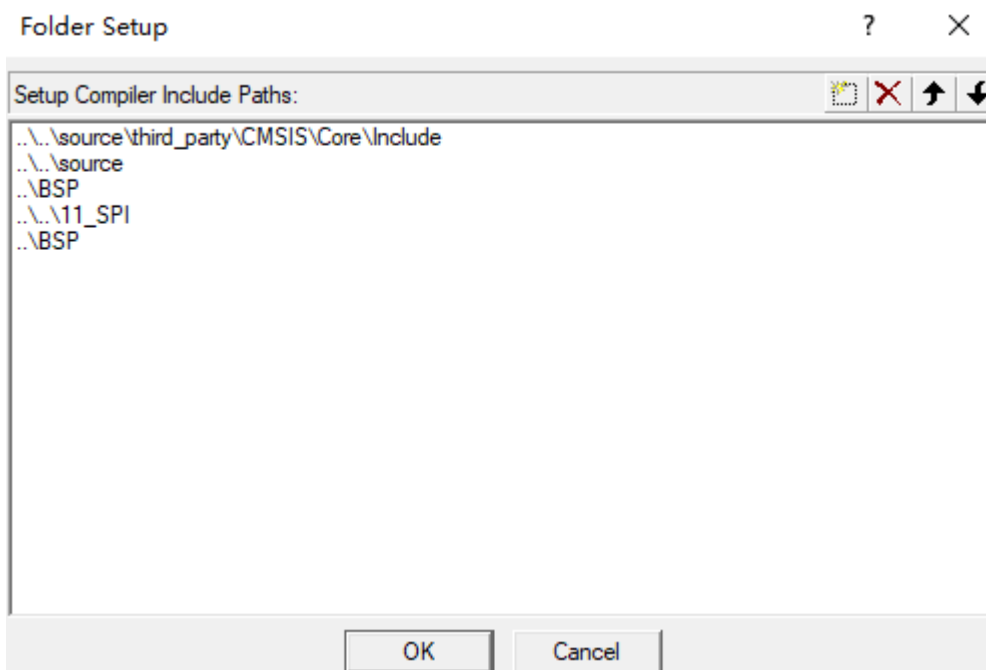
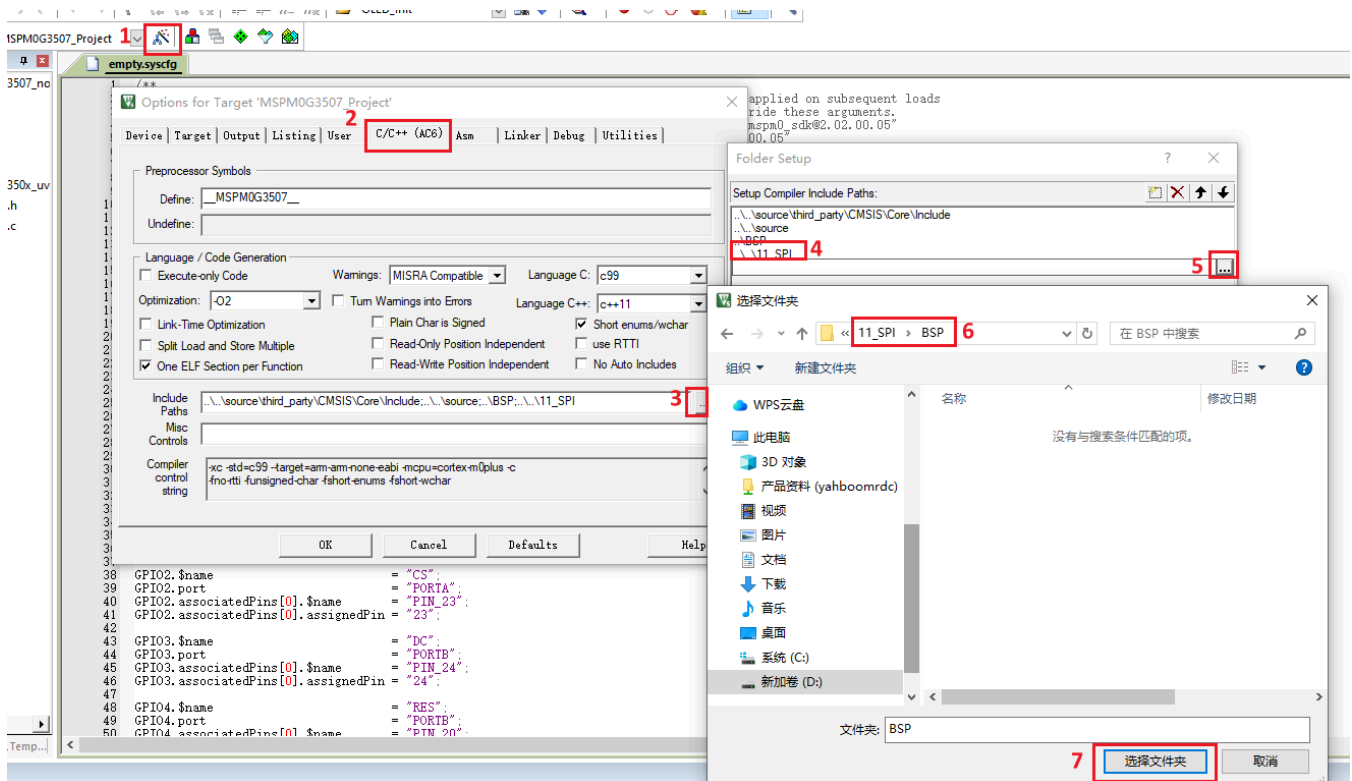
名称	修改日期	类型	大小
bsp_spi.c	2024/11/29 17:45	C 文件	5 KB
bsp_spi.h	2024/11/29 17:31	C Header 源文件	1 KB
flash_use.c	2024/11/29 17:50	C 文件	2 KB
lcd.c	2024/11/26 16:31	C 文件	11 KB
lcd.h	2024/11/26 16:06	C Header 源文件	3 KB
lcd_init.c	2024/11/26 16:31	C 文件	7 KB
lcd_init.h	2024/11/21 16:33	C Header 源文件	3 KB
lcdfont.h	2023/1/9 16:23	C Header 源文件	72 KB
pic.h	2023/1/9 22:57	C Header 源文件	52 KB

Character library

Open project management, click BSP, and add the files we previously created and copied to BSP.



Update header file paths.



4. Write Program

Define LCD screen operation interface and related functions

lcd.h

```
#ifndef __LCD_H
#define __LCD_H
#include <stdint.h>

void LCD_Fill(uint16_t xsta,uint16_t ysta,uint16_t xend,uint16_t yend,uint16_t
color);//Specify area fill color
```

```

void LCD_DrawPoint(uint16_t x,uint16_t y,uint16_t color);//Draw a point at the specified
location
void LCD_DrawLine(uint16_t x1,uint16_t y1,uint16_t x2,uint16_t y2,uint16_t color);//Draw a
line at the specified position
void LCD_DrawRectangle(uint16_t x1, uint16_t y1, uint16_t x2, uint16_t y2,uint16_t
color);//Draw a rectangle at the specified location
void Draw_Circle(uint16_t x0,uint16_t y0,uint8_t r,uint16_t color);//Draw a circle at the
specified location

void LCD_ShowChinese(uint16_t x,uint16_t y,uint8_t *s,uint16_t fc,uint16_t bc,uint8_t
sizey,uint8_t mode);//Display Chinese character string
void LCD_ShowChinese12x12(uint16_t x,uint16_t y,uint8_t *s,uint16_t fc,uint16_t bc,uint8_t
sizey,uint8_t mode);//Display a single 12x12 Chinese character
void LCD_ShowChinese16x16(uint16_t x,uint16_t y,uint8_t *s,uint16_t fc,uint16_t bc,uint8_t
sizey,uint8_t mode);//Display a single 16x16 Chinese character
void LCD_ShowChinese24x24(uint16_t x,uint16_t y,uint8_t *s,uint16_t fc,uint16_t bc,uint8_t
sizey,uint8_t mode);//Display a single 24x24 Chinese character
void LCD_ShowChinese32x32(uint16_t x,uint16_t y,uint8_t *s,uint16_t fc,uint16_t bc,uint8_t
sizey,uint8_t mode);//Display a single 32x32 Chinese character

void LCD_ShowChar(uint16_t x,uint16_t y,uint8_t num,uint16_t fc,uint16_t bc,uint8_t
sizey,uint8_t mode);//Display a character
void LCD_ShowString(uint16_t x,uint16_t y,const uint8_t *p,uint16_t fc,uint16_t bc,uint8_t
sizey,uint8_t mode);//Display String
uint32_t mypow(uint8_t m,uint8_t n);//Power
void LCD_ShowIntNum(uint16_t x,uint16_t y,uint16_t num,uint8_t len,uint16_t fc,uint16_t
bc,uint8_t sizey);//Display integer variables
void LCD_ShowFloatNum1(uint16_t x,uint16_t y,float num,uint8_t len,uint16_t fc,uint16_t
bc,uint8_t sizey);//Display variables with two decimal places

void LCD_ShowPicture(uint16_t x,uint16_t y,uint16_t length,uint16_t width,const uint8_t
pic[]);//Show image

//Brush Color
#define WHITE          0xFFFF
#define BLACK          0x0000
#define BLUE           0x001F
#define BRED           0xF81F
#define GRED           0xFFE0
#define GBLUE          0x07FF
#define RED            0xF800
#define MAGENTA        0xF81F
#define GREEN          0x07E0
#define CYAN           0x7FFF
#define YELLOW         0xFFE0
#define BROWN          0xBC40 //Brown
#define BRRED          0xFC07 //Brown red
#define GRAY           0x8430 //Gray
#define DARKBLUE       0x01CF //Dark blue
#define LIGHTBLUE      0x7D7C //Light blue
#define GRAYBLUE       0x5458 //Gray blue
#define LIGHTGREEN     0x841F //Light green

```

```

#define LGRAY          0XC618 //Light gray (PANNEL), window background color
#define LGRAYBLUE      0XA651 //Light gray blue (middle layer color)
#define LBBLUE         0X2B12 //Light brown blue (inverted color of selected item)

#endif

```

Next is the LCD screen driver

lcd.c (only partial content is shown here, see project source code for details)

```

#include "lcd.h"
#include "lcd_init.h"
#include "lcdfont.h"
#include "board.h"

/*****
    Function description: Fill the specified area with color
    Input data: xsta, ysta starting coordinates
                xend, yend ending coordinates
                color the color to be filled
    Return value: None
*****/
void LCD_Fill(uint16_t xsta,uint16_t ysta,uint16_t xend,uint16_t yend,uint16_t color)
{
    uint16_t i,j;
    LCD_Address_Set(xsta,ysta,xend-1,yend-1);//Set the display range
    for(i=ysta;i<yend;i++)
    {
        for(j=xsta;j<xend;j++)
        {
            LCD_WR_DATA(color);
        }
    }
}

/*****
    Function description: Draw a point at the specified position
    Input data: x, y coordinates of the point
                color color of the point
    Return value: None
*****/
void LCD_DrawPoint(uint16_t x,uint16_t y,uint16_t color)
{
    LCD_Address_Set(x,y,x,y);//Set the cursor position
    LCD_WR_DATA(color);
}

...

```

bsp_spi.h

```

#ifndef _BSP_SPI_H__

```

```

#define _BSP_SPI_H__

#include "board.h"

//CS pin output control
//x=0 when output is low level
//x=1 when output is high level
#define SPI_CS(x) ( (x) ? DL_GPIO_setPins(CS1_PORT,CS1_PIN_17_PIN) :
DL_GPIO_clearPins(CS1_PORT,CS1_PIN_17_PIN) )

uint16_t w25q32_readID(void); //Read the ID of W25Q32
void w25q32_write(uint8_t* buffer, uint32_t addr, uint16_t numbyte); //w25q32 write
data
void w25q32_read(uint8_t* buffer, uint32_t read_addr, uint16_t read_length); //w25q32 Read Data
#endif

```

SPI initialization has been configured in SYSCONFIG, but we still need to prepare SPI read/write steps. To ensure successful sending and receiving of data, when sending, you need to ensure the data in the send buffer has been sent completely, i.e., the send buffer is empty before you can send the next data; when receiving, you need to ensure there is data in the receive buffer before you can receive.

bsp_spi.c (only partial content is shown here, see project source code for details)

```

#include "bsp_spi.h"

uint8_t spi_read_write_byte(uint8_t dat)
{
    uint8_t data = 0;

    //Sending Data
    DL_SPI_transmitData8(SPI_INST, dat);
    //Wait for the SPI bus to be idle
    while(DL_SPI_isBusy(SPI_INST));
    //Receiving Data
    data = DL_SPI_receiveData8(SPI_INST);
    //Wait for the SPI bus to be idle
    while(DL_SPI_isBusy(SPI_INST));

    return data;
}

...

```

Then in the empty.c file, write the following code

```

#include "ti_msp_dl_config.h"
#include "board.h"
#include "lcd_init.h"
#include "lcd.h"
#include "pic.h"
#include "bsp_spi.h"

```

```

#include <stdint.h>
#include <stdio.h>

int main(void)
{
    unsigned char buff[10] = {0};

    //Development board initialization
    board_init();

    delay_ms(100); //waiting for deployment

    //Read the ID of W25Q32
    printf("ID = %X\r\n", W25Q32_readID());

    //Read 5 bytes of data from address 0 to buff
    W25Q32_read(buff, 0, 7);

    //Serial port outputs the read data
    printf("buff = %s\r\n", buff);

    //Write 5 bytes of data ABCD to address 0
    W25Q32_write("Yahboom", 0, 7);
    delay_ms(100);

    //Read 5 bytes of data from address 0 to buff
    W25Q32_read(buff, 0, 7);

    //Serial port outputs the read data
    // printf("buff = %s\r\n", buff);

    SYSCFG_DL_init();

    LCD_Init(); //LCD Initialization
    LCD_Fill(0, 0, LCD_W, LCD_H, WHITE);
    LCD_ShowPicture(20, 45, 120, 29, gImage_pic1);
    LCD_ShowString(10, 0, "Hello!", BLACK, WHITE, 16, 0);
    LCD_ShowChinese(50, 20, "亚博智能", BLACK, WHITE, 16, 0);

    // Display the contents of buff on the same line
    int x = 54; // Starting x coordinate
    for (int i = 0; i < 7; i++)
    {
        char str[2] = {buff[i], '\0'}; // Take one character at a time
        LCD_ShowString(x, 36, str, BLACK, WHITE, 16, 0); // Display Characters
        x += 8; // Each character is 8 wide and has an x-coordinate offset to the right.
    }

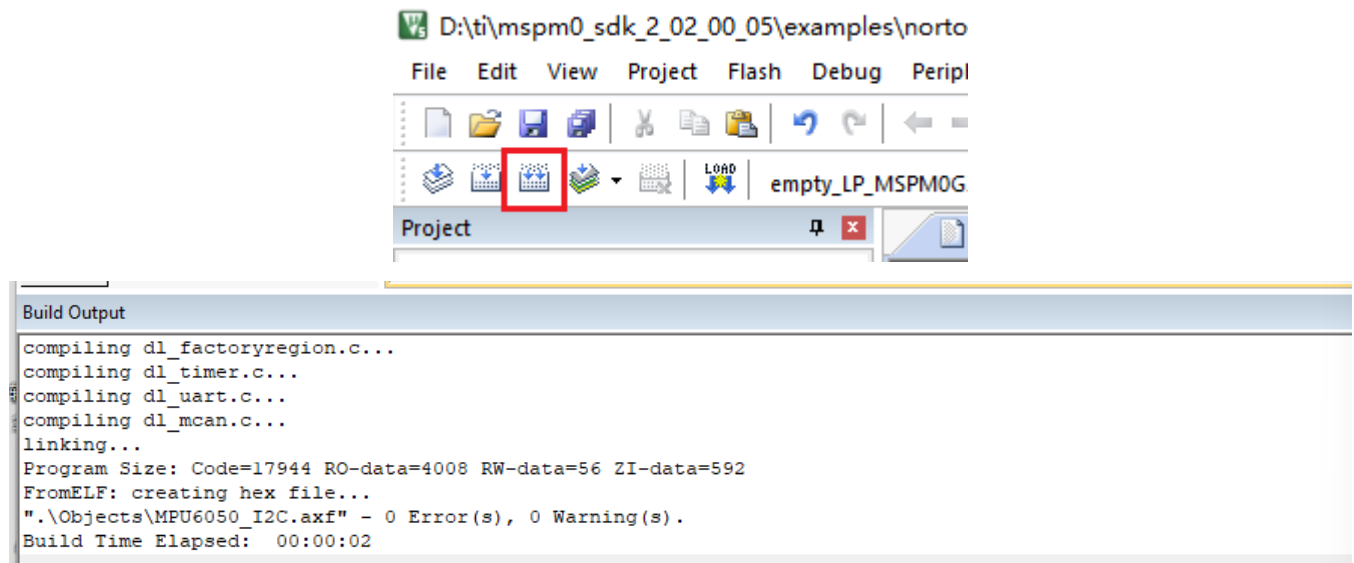
    while (1)
    {

```

```
}
```

5. Compile

Click the Rebuild icon. The following prompt indicates compilation is complete with no errors.



4. Program Analysis

- lcd.c

```
/*
*****
Function description: Fill the specified area with color
Input data: xsta, ysta starting coordinates
             xend, yend ending coordinates
             color the color to be filled
Return value: None
*****
*/
void LCD_Fill(uint16_t xsta,uint16_t ysta,uint16_t xend,uint16_t yend,uint16_t color)
{
    uint16_t i,j;
    LCD_Address_Set(xsta,ysta,xend-1,yend-1);//Set the display range
    for(i=ysta;i<yend;i++)
    {
        for(j=xsta;j<xend;j++)
        {
            LCD_WR_DATA(color);
        }
    }
}
```

The `LCD_Fill` function fills the specified area from starting coordinates (`xsta`, `ysta`) to ending coordinates (`xend`, `yend`) with the specified color by setting the display range and writing color data in a loop.

```
/*
*****
Function description: Display image
Input data: x, y starting point coordinates
*****
*/
```

```

        length Image length
        width Image width
        pic[] Image array

    Return value: None
    *****/
void LCD_ShowPicture(uint16_t x,uint16_t y,uint16_t length,uint16_t width,const uint8_t
pic[])
{
    uint16_t i,j;
    uint32_t k=0;
    LCD_Address_Set(x,y,x+length-1,y+width-1);
    for(i=0;i<length;i++)
    {
        for(j=0;j<width;j++)
        {
            LCD_WR_DATA8(pic[k*2]);
            LCD_WR_DATA8(pic[k*2+1]);
            k++;
        }
    }
}

```

The `LCD_ShowPicture` function draws an image at the specified starting point `(x, y)`. It sets the display range according to the given length and width, and by traversing the image array, writes each pixel's color data to the LCD, thus completing image display.

```

/*****
    Function description: Display string
    Input data: x, y display coordinates
                *p string to be displayed
                fc color of the word
                bc background color of the word
                sizey font size
                mode: 0 non-overlay mode 1 overlay mode

    Return value: None
    *****/
void LCD_ShowString(uint16_t x,uint16_t y,const uint8_t *p,uint16_t fc,uint16_t bc,uint8_t
sizey,uint8_t mode)
{
    while(*p!='\0')
    {
        LCD_ShowChar(x,y,*p,fc,bc,sizey,mode);
        x+=sizey/2;
        p++;
    }
}

```

The `LCD_showString` function displays a string at the specified coordinates `(x, y)`. It calls the `LCD_ShowChar` function to draw each character in the string, adjusts character spacing according to font size, and supports both overlay and non-overlay display modes.

```

/*****
Function description: Display Chinese character string
Input data: x, y display coordinates
            *s Chinese character string to be displayed
            fc character color
            bc character background color
            sizey font size optional 16 24 32
            mode: 0 non-overlay mode 1 overlay mode

Return value: None
*****/
void LCD_ShowChinese(uint16_t x,uint16_t y,uint8_t *s,uint16_t fc,uint16_t bc,uint8_t
sizey,uint8_t mode)
{
    while(*s!=0)
    {
        LCD_ShowChinese16x16(x,y,s,fc,bc,sizey,mode);
        s+=2;
        x+=sizey;
    }
}

```

The `LCD_ShowChinese` function displays a Chinese character string at the specified coordinates `(x, y)`. It draws each Chinese character by calling the `LCD_ShowChinese16x16` function, and adjusts the next Chinese character's display position according to font size, supporting both overlay and non-overlay display modes.

- bsp_spi.c

```

/*****
* Function name: W25Q32_read
* Function function: Read W25Q32 data
* Input parameters: buffer = storage address of read data read_addr = read address
read_length = read length
* Function return: None
* Author: LC
* Notes: None
*****/
void W25Q32_read(uint8_t* buffer,uint32_t read_addr,uint16_t read_length)
{
    uint16_t i;
    //Pull the CS end to a low level
    SPI_CS(0);
    //Send instruction 03h
    spi_read_write_byte(0x03);
    //Send the high 8 bits of the 24-bit read data address
    spi_read_write_byte((uint8_t)((read_addr)>>16));
    //Send the middle 8 bits of the 24-bit read data address
    spi_read_write_byte((uint8_t)((read_addr)>>8));
    //Send the low 8 bits of the 24-bit read data address
    spi_read_write_byte((uint8_t)read_addr);
    //Read the address according to the read length and save it in the buffer
    for(i=0;i<read_length;i++)
    {
        buffer[i]= spi_read_write_byte(0xFF);
    }
}

```

```

    }
    //Restore the CS end to a high level
    SPI_CS(1);
}

```

The `w25q32_read` function reads data from the W25Q32 flash chip. The function first starts SPI communication by pulling the CS pin low, then sends the read command (0x03) and the high, middle, and low bytes of the read address. Then, according to the specified read length, it reads data byte by byte and stores it in the provided buffer (`buffer`). Finally, the function restores the CS pin to high level, ending the data transfer. This process ensures reading the specified length of data from the specified address.

```

/*****
 * Function name: w25q32_write
 * Function function: Write data to w25q32 for storage
 * Input parameters: buffer = data content to be written addr = write address numbyte =
length of written data
 * Function return: None
 * Author: LC
 * Notes: None
 *****/
void w25q32_write(uint8_t* buffer, uint32_t addr, uint16_t numbyte)
{
    unsigned int i = 0;
    //Erase sector data
    w25q32_erase_sector(addr/4096);
    //write enable
    w25q32_write_enable();
    //Busy detection
    w25q32_wait_busy();
    //write data
    //Pull CS to low level
    SPI_CS(0);
    //Send instruction 02h
    spi_read_write_byte(0x02);
    //Send the high 8 bits of the 24-bit address to be written
    spi_read_write_byte((uint8_t)((addr)>>16));
    //Send the middle 8 bits of the 24-bit address to be written
    spi_read_write_byte((uint8_t)((addr)>>8));
    //Send the low 8 bits of the 24-bit address to be written
    spi_read_write_byte((uint8_t)addr);
    //Continuously write data buffer according to the length of the written byte
    for(i=0;i<numbyte;i++)
    {
        spi_read_write_byte(buffer[i]);
    }
    //Restore CS end to high level
    SPI_CS(0);
    //Busy detection
    w25q32_wait_busy();
}

```

The `w25q32_write` function writes data to the W25Q32 flash chip. First, the function erases the sector data at the target address. Then, it enables write operation and checks if the chip is busy. Next, the function sends the write command (0x02) and data address (24-bit address) to the chip via SPI interface, and writes data byte by byte according to the specified write length. Finally, the function waits for the write operation to complete and restores the CS pin to high level, ensuring data is correctly written and the chip is in idle state.

- empty.c

```
int main(void)
{
    unsigned char buff[10] = {0};

    //Development board initialization
    board_init();

    delay_ms(100); //waiting for deployment

    //Read the ID of w25q32
    printf("ID = %X\r\n", w25q32_readID());

    //Read 5 bytes of data from address 0 to buff
    w25q32_read(buff, 0, 7);

    //Serial port outputs the read data
    printf("buff = %s\r\n", buff);

    //Write 5 bytes of data ABCD to address 0
    w25q32_write("Yahboom", 0, 7);
    delay_ms(100);

    //Read 5 bytes of data from address 0 to buff
    w25q32_read(buff, 0, 7);

    //Serial port outputs the read data
    // printf("buff = %s\r\n", buff);

    SYSCFG_DL_init();

    LCD_Init(); //LCD Initialization
    LCD_Fill(0, 0, LCD_W, LCD_H, WHITE);
    LCD_ShowPicture(20, 45, 120, 29, gImage_pic1);
    LCD_ShowString(10, 0, "Hello!", BLACK, WHITE, 16, 0);
    LCD_ShowChinese(50, 20, "亚博智能", BLACK, WHITE, 16, 0);

    // Display the contents of buff on the same line
    int x = 54; // Starting x coordinate
    for (int i = 0; i < 7; i++)
    {
        char str[2] = {buff[i], '\0'}; // Take one character at a time
        LCD_ShowString(x, 36, str, BLACK, WHITE, 16, 0); // Display Characters
        x += 8; // Each character is 8 wide and has an x-coordinate offset to the right.
    }
}
```

```
while (1)
{

}
}
```

The main function of this program is to initialize the development board, read and write data to the W25Q32 flash chip, and display the read content on the LCD. First, the program initializes the development board and related hardware. Then, it reads the W25Q32 chip ID and outputs it via serial port. Next, the program reads 7 bytes of data from the flash's starting address (address 0) into the `buff` array and outputs this data via serial port. Then, it writes the string "Yahboom" to the flash's address 0, and reads and displays the data again. Finally, the program initializes the LCD screen, displays welcome information and images, and displays the read `buff` data character by character on the same line of the screen.

5. Experimental Phenomenon

After the program is downloaded, the LCD display screen will show the text "Hello!", "亚博智能", data in flash, and Yahboom's logo on separate lines.

